

RELATÓRIO TÉCNICO: ANÁLISE COMPARATIVA DE CLASSIFICAÇÃO SUPERVISIONADA EM R E PYTHON PARA PREVISÃO DE CHURN

1. Introdução

A retenção de clientes tornou-se um dos pilares estratégicos mais críticos para a sustentabilidade financeira de empresas no setor de telecomunicações. O fenómeno do *churn* (taxa de cancelamento de clientes) impacta diretamente a receita previsível e eleva o custo de aquisição de novos clientes (CAC). Diante deste cenário, a aplicação de técnicas de Ciência de Dados para antecipar o comportamento de cancelamento permite que as organizações tomem medidas proativas de retenção.

O objetivo central deste projeto aplicado é desenvolver uma solução comparativa de classificação supervisionada utilizando os algoritmos Naive Bayes, Árvore de Decisão e Random Forest. A análise avalia o desempenho preditivo dos modelos e o comportamento das implementações equivalentes nas linguagens R e Python, mitigando vieses através do uso de partições de dados estritamente idênticas.

2. Fundamentação Teórica

Algoritmos de Classificação Supervisionada

- **Naive Bayes:** É um classificador probabilístico baseado no Teorema de Bayes que assume a premissa de independência condicional entre todos os atributos explicativos (daí o termo "ingênuo"). Apesar desta simplificação teórica, apresenta excelente eficiência computacional e robustez em problemas de alta dimensionalidade.
- **Árvore de Decisão:** Algoritmo não-paramétrico que particiona os dados recursivamente com base em regras lógicas condicionais (if-then-else). A divisão dos nós baseia-se em métricas de pureza, como o Índice Gini ou a Entropia. Destaca-se pela sua alta interpretabilidade visual.
- **Random Forest:** Um método de aprendizagem por conjunto (*ensemble*) que constrói uma "floresta" composta por múltiplas árvores de decisão independentes treinadas com subconjuntos aleatórios dos dados (técnicas de *bagging* e *feature sampling*). O resultado final é gerado por voto majoritário, reduzindo drasticamente o risco de *overfitting* inerente às árvores isoladas.

3. Metodologia

Descrição da Base de Dados e Pré-processamento

A solução utiliza a base de dados pública *Telco Customer Churn*. A variável alvo categórica é o Churn (indica se o cliente cancelou o serviço: "Yes" ou "No"),

acompanhada por 19 atributos explicativos que cobrem dados demográficos, serviços contratados e informações financeiras.

O pré-processamento seguiu as seguintes etapas técnicas:

1. **Eliminação de Identificadores:** A coluna `customerID` foi removida por não possuir valor preditivo, evitando o sobreajuste do modelo.
2. **Tratamento de Dados Ausentes:** Foram identificados e eliminados 11 registros nulos na variável `TotalCharges`. Dada a dimensão amostral (>7.000 registros), a exclusão destas linhas minimizou impactos estatísticos sem introduzir o viés de imputações sintéticas.
3. **Codificação de Variáveis:** No ambiente R, as strings foram tratadas nativamente como fatores explicativos. No Python, utilizou-se a técnica de codificação *One-Hot Encoding* (`pd.get_dummies`), necessária para a conformidade matemática da biblioteca *Scikit-Learn*.

Estratégia Experimental e Critérios de Avaliação

Para garantir a reprodutibilidade e a justiça no processo comparativo, fixou-se a semente aleatória (`set.seed(42)` / `random_state=42`). A base foi dividida de forma estratificada em 80% para treino (5.627 observações) e 20% para teste (1.405 observações).

As bibliotecas empregues foram `caret`, `e1071`, `rpart` e `randomForest` no ecossistema R, e `pandas` e `scikit-learn` no ambiente Python. Os modelos foram avaliados com base em matrizes de confusão e nas métricas de Acurácia, Precisão, Revocação (*Sensitivity*) e F1-score.

4. Implementação

O desenvolvimento foi dividido em duas frentes para atender ao critério de equivalência metodológica:

R: Foi utilizado o RStudio como ambiente de desenvolvimento. A divisão dos dados foi realizada através do pacote `caret`, utilizando a função `createDataPartition` com a semente aleatória `set.seed(42)` para estratificação. Os modelos foram treinados utilizando os pacotes `e1071` (Naive Bayes), `rpart` (Árvore de Decisão) e `randomForest` (Random Forest). Para garantir a integridade da comparação subsequente, as partições exatas de treino e teste geradas pelo R foram exportadas em formato CSV.

Python: Foi utilizado a biblioteca `pandas` para a importação das exatas partições CSV geradas no R, garantindo que ambos os ambientes avaliassem as mesmas observações. Devido a biblioteca `scikit-learn`, aplicou-se a técnica de *One-Hot Encoding* (`pd.get_dummies`) para a conversão de atributos textuais em numéricos 1 e 0. Os modelos foram instanciados utilizando `GaussianNB`, `DecisionTreeClassifier` e `RandomForestClassifier`. Como exigência adicional do projeto, aplicou-se a técnica de ajuste de hiperparâmetros (*Tuning*) através do `GridSearchCV` na Árvore de Decisão.

5. Resultados

As matrizes de confusão detalhadas e as métricas individuais de cada configuração encontram-se documentadas no apêndice deste relatório. A Tabela 1 resume o desempenho preditivo dos algoritmos nas duas linguagens sobre o conjunto de teste (1.405 observações).

Algoritmo	Linguagem	Acurácia	Precisão	Revocação	F1-Score
Naive Bayes	R	73,67%	50,25%	80,43%	61,86%
Naive Bayes	Python	67,62%	44,40%	87,13%	58,82%
Árvore de Decisão	R	78,65%	68,15%	36,72%	47,73%
Árvore de Decisão (Base)	Python	73,52%	50,13%	52,01%	51,05%
Árvore de Decisão (Melhorada)	Python	78,72%	67,45%	38,34%	48,88%
Random Forest	R	78,86%	64,07%	46,38%	53,81%
Random Forest	Python	78,86%	64,73%	44,77%	52,93%

6. Discussão Crítica

Comportamento dos Algoritmos:

O Random Forest apresentou a maior estabilidade técnica, cravando exatos 78,86% de acurácia em ambas as linguagens, confirmando a robustez do *ensemble* em mitigar ruídos independentemente da engine de processamento.

A Árvore de Decisão, no Python, sem restrições nativas de profundidade, o modelo sofreu sobreajuste, resultando numa acurácia menor (73,52%). Contudo, ao aplicar o GridSearchCV (configurando a profundidade máxima `max_depth=3`), a acurácia saltou para 78,72%, equiparando-se ao comportamento "podado" nativamente pelo pacote `rpart` do R.

O Naive Bayes revelou-se o modelo mais agressivo. No Python, devido à transformação matemática contínua das variáveis *dummies*, o algoritmo "sacrificou" a precisão global para alcançar uma notável revocação de 87,13%, identificando quase todos os clientes insatisfeitos (Verdadeiros Positivos), ao custo de gerar alarmes falsos (Falsos Positivos).

7. Referências

- <https://www.kaggle.com/datasets/blastchar/telco-customer-churn?resource=download>